

# An Optimal Vector Clock Algorithm for Multithreaded Systems

1<sup>st</sup> Xiong Zheng

Electrical and Computer Engineering  
The University of Texas  
Austin, USA  
zhengxiongty@utexas.edu

2<sup>nd</sup> Vijay K. Garg

Electrical and Computer Engineering  
The University of Texas  
Austin, USA  
garg@ece.utexas.edu

**Abstract**—Tracking causality (or happened-before relation) between events is useful for many applications such as debugging and recovery from failures. Consider a concurrent system with  $n$  threads and  $m$  objects. For such systems, either a vector clock of size  $n$  is used with one component per thread or a vector clock of size  $m$  is used with one component per object. A natural question is whether one can use a vector clock of size strictly less than the minimum of  $m$  and  $n$  to timestamp events. We give an algorithm in this paper that uses a hybrid of thread and object components. Our algorithm is guaranteed to return the minimum number of components necessary for vector clocks. We first consider the case when the interaction between objects and threads is statically known. This interaction is modeled by a thread-object bipartite graph. Our algorithm is based on finding the maximum bipartite matching of such a graph and then applying König-Egerváry Theorem to compute the minimum vertex cover to determine the optimal number of components necessary for the vector clock. We also propose two mechanisms to compute such an vector clock when computation is revealed in an online fashion. Evaluation on different types of graphs indicates that our offline algorithm generates a size vector clock which is significantly less than the minimum of  $m$  and  $n$ . These mechanisms are more effective when the underlying bipartite graph is not dense.

**Index Terms**—optimal vector clocks, bipartite matching

## I. INTRODUCTION

A fundamental problem in parallel and distributed systems is to determine the order relationship between events of a distributed computation as defined by Lamport's *happened-before relation* [9]. The problem arises in many areas including debugging and visualization of parallel and distributed programs.

Vector clocks, which were introduced independently by Fidge [4]–[6] and Mattern [11], and their variants [10] are widely used to capture the causality between events in parallel and distributed systems. To capture the causality, each event is timestamped with the current value of the local vector clock at the time the event is generated. The order relationship between two events can then be determined by comparing their timestamps. A vector clock contains one component for every process in a distributed system. This results in message and space overhead of  $N$  integers in a distributed system consisting of  $N$  processes. In shared-memory based systems, there are

two kinds of vector clocks. Consider a concurrent system with  $n$  threads and  $m$  objects. For such systems, either vector clocks of size  $n$  is used with one component per thread or a vector clock of size  $m$  is used with one component per object. A natural question is whether one can use a vector of size strictly less than the minimum of  $m$  and  $n$  to timestamp events. We show that this is indeed possible in this paper.

Consider the example in Fig. 1 to understand why it is feasible to use a smaller vector clock to order events. In this example,  $T_1, T_2, T_3, T_4$  are threads,  $O_1, O_2, O_3, O_4$  are objects which are used by the above threads. Each circle represents an operation. To order those operations, traditionally, either all threads or all objects are exploited as components of vector clock so as to indicate the causal order. However, notice that all the operations are related to either thread  $T_2$  or object  $O_2$  or object  $O_3$ . Therefore, we can use a vector clock composed of  $T_2, O_2$ , and  $O_3$  to timestamp all events. This mixed-vector-clock has the size of 3 which is smaller than the number of threads and the number of objects. This example answers the question that a vector of size strictly less than minimum of  $m$  and  $n$  can be obtained to timestamp a computation.

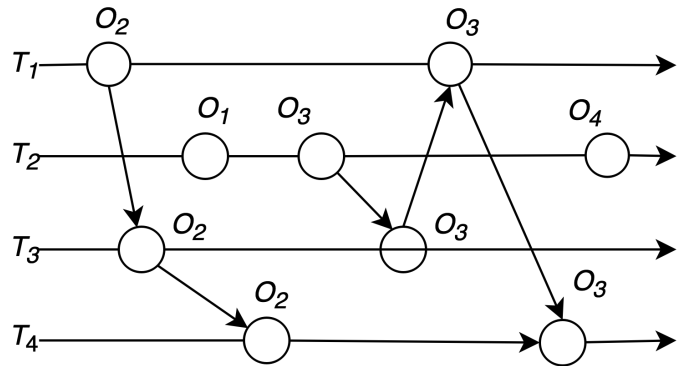


Fig. 1. A computation of threads operating on shared objects

A computation of threads operating on objects can be represented as a bipartite graph and the minimum vertex cover of a bipartite graph is no greater than the minimum of the number of left nodes and number of right nodes. Recall the

definition of a vertex cover of a graph: a set of vertexes such that each edge in the graph is incident to at least one vertex of the set and notice that an edge in a bipartite graph actually corresponds to an event in a computation. Therefore, by converting such a computation to a thread-object bipartite graph (to be introduced in section III), we can make use of the minimum vertex cover of this bipartite graph to timestamp the events in the computation. Thus, the minimum vertex cover is used to determine the components of the vector clock for this computation. On this basis, we give an offline algorithm that makes use of a hybrid of thread and object components which is smaller in size than traditional vector clocks to timestamp a computation. Our offline algorithm is guaranteed to return the minimum number of components necessary for a vector clock. This algorithm is based on finding a maximum bipartite matching in the thread object bipartite graph and then applying König-Egerváry Theorem [1] to determine the optimal number of components necessary for the vector clock.

We also consider the case when the interaction between threads and objects is not known *a priori*. We propose two mechanisms to address this problem and compare performance with the traditional solution.

In summary, this paper makes the following contributions:

- We introduce the notion of a mixed-vector-clock that satisfies the vector clock condition with fewer than thread-based or object-based clock.
- We give an optimal algorithm to determine which threads and objects should be used for a mixed-vector-clock.
- We give two mechanisms to compute the mixed-vector-clock when events of a computation is coming in an online fashion.

## II. SYSTEM MODEL AND NOTATION

In this section, we present our model of a concurrent system. The system consists of  $N$  sequential processes (or threads) denoted by  $P = \{p_1, p_2, \dots, p_n\}$  performing operations on  $m$  objects denoted by  $Q = \{q_1, q_2, \dots, q_m\}$ . In the remaining section, we use threads or processes interchangeably. A *computation* in the happened before model is defined as a tuple  $(E, \rightarrow)$  where  $E$  is the set of events and  $\rightarrow$  is a partial order on events in  $E$ . Each process executes a sequence of events. Each event is performed on a single object. We assume that all operations on any single object are performed sequentially (for example, by using locks). For an event  $e \in E$ ,  $e.thread$  denotes the process on which  $e$  occurred and  $e.object$  denotes the object on which  $e$  occurred.

Then Lamport's happened-before relation ( $\rightarrow$ ) on  $E$  is the smallest transitive relation such that:

1. If  $e.thread = f.thread$  and  $e$  immediately precedes  $f$  in the sequence of events in process  $e.thread$ , then  $e \rightarrow f$ .
2. If  $e.object = f.object$  and  $e$  immediately precedes  $f$  in the sequence of events on the object  $e.q$ , then  $e \rightarrow f$ .

Two events  $e$  and  $f$  are said to be *comparable* if  $e \rightarrow f$  or  $f \rightarrow e$ . If  $e$  and  $f$  are not comparable, they are said to be *concurrent* and this relationship is denoted by  $e \parallel f$ .

We define process-object graph as the undirected bipartite graph  $G = (P, Q, T)$  where  $T$  is the set of edges between the set of processes  $P$  and the set of objects  $Q$  defined as

$$T = \{(p, q) | \text{the object } q \text{ is accessible to process } p\}$$

In most applications, we do not expect  $G$  to be dense, i.e, a process typically has references to only a small subset of objects.

The set of events  $E$  with the order imposed by Lamport's happened before relation defines a partially ordered set or *poset*. A subset of elements  $C \subseteq E$  is said to form a *chain* iff  $\forall e, f \in C : e \rightarrow f$  or  $f \rightarrow e$ . By our definition of processes all operations done by a single process form a chain. Similarly, all operations done on a single object also form a chain.

Process-based vector clocks maintain a vector  $V$  of size  $|P|$  with each process and object. Whenever, a process  $p$  executes an operation  $e$  on object  $q$  it gets the timestamp  $e.v$  as

$$e.v = \max(p.v, q.v); e.v[e.thread] ++;$$

Both  $p$  and  $q$  update their vector to  $e.v$ .

Object-based vector clocks maintain a vector  $V$  of size  $|Q|$  with each process and object. Whenever, a process  $p$  executes an operation  $e$  on object  $q$  it gets the timestamp  $e.v$  as

$$e.v = \max(p.v, q.v); e.v[e.object] ++;$$

Both  $p$  and  $q$  update their vector to  $e.v$ .

In this paper, we design a vector clock called *mixed vector-clock* that uses a combination of processes and objects for its components. Clearly, process-based and object-based vector clocks are special case of our scheme. Moreover, the total number of components in a mixed vector clock is always less than or equal to the minimum of the process and object based vector clocks.

## III. AN OFFLINE ALGORITHM

In this section, we first introduce the thread-object bipartite graph for a computation. Next, we give an offline algorithm to compute the optimal mix-vector-clock by computing the maximum bipartite matching of the thread-object bipartite graph and obtaining a minimum vertex cover. Then we show the mix-vector-clock given by this offline algorithm is optimal in terms of size.

### A. The Thread-object Bipartite Graph

A thread-object computation is composed of events which are in the form of some specific thread doing some operations on a specific object. Notice that such a computation only involves two parties: threads and objects. An operation relates an thread and an object. Therefore, such a computation could be modeled as a bipartite graph if we only focus on the relation between the two parties, i.e, for thread  $p$  and object  $q$ , we only care about whether  $p$  has any operation on  $q$  or not and ignore exactly how many operations that  $p$  has on  $q$ . If a thread has at least one operation on an object, then there is an edge between them in the bipartite graph. We call such a bipartite graph as

thread-object bipartite graph herein. The computation shown in Fig. 1 can be converted to the thread-object bipartite graph given in Fig. 2. The filled vertices represent the minimum vertex cover of this bipartite graph or the components of our *mix-vector-clock*.

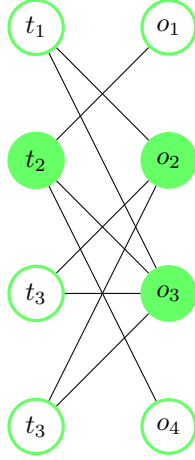


Fig. 2. Thread-Object Bipartite Graph of A Computation

Let  $G = \{T \cup O, E\}$  be the thread-object bipartite graph of a given computation.  $T$  denote the set of threads involved in the given computation.  $O$  denotes the set of objects to which threads in  $T$  have operations on in the given computation. For any thread  $t \in T$  and object  $o \in O$ , there is an edge between thread  $t$  and object  $o$  in the thread-object bipartite graph iff thread  $t$  has at least one operation on object  $o$  in the given computation.

### B. The Offline Algorithm

Assuming that the thread-object graph representing a computation is given or constructed by the trace generator, we show how to obtain our *mix-vector-clock* by computing a minimum vertex cover of this bipartite graph. In order to compute such a vertex cover, we use König-Egerváry Theorem.

**Theorem 1 (König-Egerváry Theorem):** In any bipartite graph, the maximum size of a matching equals the minimum size of a vertex cover.

Based on König-Egerváry's theorem, we can first compute a maximum matching of the thread-object bipartite graph, which has many algorithms. One simple and efficient such algorithm is the bipartite matching algorithm given by Hopcroft and Karp [3], which achieves a time complexity of  $O(n^{5/2})$ .

The basics of this matching algorithm is stated as below: at each iteration, this algorithm searches for shortest augmenting paths denoted as  $\{Q_1, Q_2, \dots, Q_t\}$  relative to existing matching  $M$  and augment the current matching. The new matching  $M'$  is obtained by  $M \oplus Q_1 \oplus Q_2 \oplus \dots \oplus Q_t$ , where  $\oplus$  represents symmetric difference. When there is no augmenting path in the bipartite graph, the maximum matching is found. The details of this algorithm can be found in [3].

Next, given the maximum matching, we can directly apply König-Egerváry Theorem to convert the maximum matching

to minimum vertex cover so as to get the *mix-vector-clock*. The pseudocode to compute the *mix-vector-clock* is given in **Algorithm 1**.

In this algorithm, we first compute a maximum matching  $M^*$  of  $G$ . Given  $M^*$ , the set of unmatched threads can be directly obtained, denote this set as  $S$ . Line 3-9 is the procedure to convert the maximum matching  $M^*$  to a minimum vertex cover.  $Z$  is the set of nodes in the graph which are connected by  $M^*$ -alternating paths to  $S$ . The minimum vertex cover  $C^*$  can be computed as  $(T - Z) \cup (O \cap Z)$ . This procedure can be found in the proof for König-Egerváry Theorem in [1] (Theorem 5.3). For completeness, we include the proof here.

---

#### Algorithm 1 Minimum *Mixed-vector-clock*

---

- 1: Finding a maximum matching of  $G$ , denoted as  $M^*$
  - 2: Compute the set of unmatched threads, denoted as  $S$
  - 3: Let  $Z := S$
  - 4: **for**  $s \in S$  **do**
  - 5:   Start from  $s$ , BFS search via alternating paths
  - 6:   Let  $B_s$  denote the set of vertexes traversed by BFS
  - 7:    $Z := Z \cup B_s$
  - 8: **end for**
  - 9: Return  $C^* = (T - Z) \cup (O \cap Z)$
- 

**Lemma 1:** **Algorithm 1** computes a minimum vertex cover of a graph.

*Proof:* First, let's show that  $C^*$  is a vertex cover. Let  $(t, o)$  be any edge from  $E$ . There are two possible cases.

Case 1: Edge  $(t, o)$  belongs to an alternating path  $p \in P$ , then it's right endpoint is in  $C$ .

Case 2: Edge  $(t, o)$  does not belong to any alternating path. If  $(t, o)$  is matched, then it's left endpoint  $t$  couldn't be in any alternating path (otherwise  $(t, o)$  belongs to such an alternating path. Thus,  $t \in (T - L)$ . If  $(t, o)$  is unmatched, then its left endpoint  $t$  cannot be in any alternating path, for such a path could be extended by adding  $(t, o)$  to it.

Second, let's prove  $C^*$  is minimum. Every vertex in  $C^*$  is matched. For, every vertex in  $(T - L)$  is matched because  $L$  is a superset of  $S$ , the set of unmatched left vertices. And every vertex in  $(O \cap L)$  must also be matched, for if there existed an alternating path to an unmatched vertex then changing the matching by removing the matched edges from this path and adding the unmatched edges in their place would increase the size of the matching. However, no matched edge can have both of its endpoints in  $C^*$ . Thus,  $C^*$  is a vertex cover of cardinality equal to  $M$ , and must be a minimum vertex cover. ■

Given the minimum vertex cover of the thread-object bipartite graph, the *mix-vector-clock* is simply constructed by assigning each thread or object in the minimum vertex cover as a component in the vector clock.

### C. Timestamping Events Using *Mix-vector-clock*

The offline algorithm gives a *mix-vector-clock*. To timestamp events in a thread object computation, we let each thread and each object keep a *mix-vector-clock*. Now let us look

at how each thread modify its mix-vector-clock in order to track causality of operations. For thread  $p$ , while performing operation  $e$  on object  $q$ , thread  $p$  needs to check whether itself or object  $q$  is in the mix-vector-clock and increases the component correspondingly. Here is how to update the timestamp of event  $e$ :

**if**  $q \in v$  **then** :  $e.v = \max(p.v, q.v); e.v[e.object] ++;$

**if**  $p \in v$  **then** :  $e.v = \max(p.v, q.v); e.v[e.thread] ++;$

Both thread  $p$  and object  $q$  update their mix-vector-clock to be  $e.v$ .

Fig. 3 shows the timestamp for each event in the computation given in Fig. 1. The components in the mix-vector-clock correspond to thread  $T_2$ , object  $O_2$  and object  $O_3$ , respectively. Initially, the *mix-vector-clock* for all threads and objects are  $[0, 0, 0]$ . Let  $[p, q]$  represents the event that thread  $p$  performs an operation on object  $q$ . In Fig. 3, the mix-vector-clock of  $[T_2, O_1]$  is smaller than vector of  $[T_3, O_3]$ , so we have  $[T_2, O_1] \rightarrow [T_3, O_3]$ . From the computation, we know that  $[T_2, O_1] \rightarrow [T_2, O_3]$  and  $[T_2, O_3] \rightarrow [T_3, O_3]$ , we also have that  $[T_2, O_1] \rightarrow [T_3, O_3]$ .

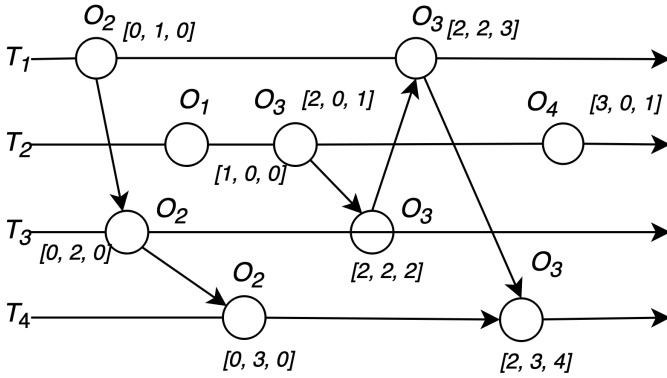


Fig. 3. Timstamping Events Using Mix-vector-clock

#### D. Proof of Correctness and Optimality

Let  $e$  be an operation in the computation,  $e.p$  be the associate process of  $e$ ,  $e.q$  be the associate object of  $e$ ,  $e.c$  be the component which is in the mix-vector-clock.  $e.c = e.p \wedge e.q$ . Let  $s$  and  $t$  be any two operations in the computation. We show the correctness and optimality by the following Lemma and Theorems.

**Lemma 2:** Let  $s \neq t$ . Then,

$$s \rightarrow t \Rightarrow t.v[s.c] < s.v[s.c]$$

**Proof:** If  $s.p = t.p$ , then it follows that  $t \prec s$ . Before executing operation  $s$ , the vector clock kept in process  $p$  must be no less than  $t.v$ , since  $p$  updates its vector to be the same as  $t.v$  after operation  $t$ . At operation  $s$ ,  $s.v = \max(s.p.v, s.q.v)$  and  $s.v[s.c] ++$ , thus  $t.v[s.c] < s.v[s.c]$ . Otherwise,  $s.p \neq t.p$ . Since  $s$  will increase  $s.v[s.c]$  will increase its component by at least one and this increase

could not have been seen by  $t$ , since  $s \rightarrow t$ , it follows that  $t.v[s.c] < s.v[s.c]$ . ■

**Theorem 2: (Correctness:)** The *mix-vector-clock* is a valid vector clock.

**Proof:** In order to show the *mix-vector-clock* is a valid vector clock, we need to show that it satisfies the constraint:  $\forall s, t : s \rightarrow t \Leftrightarrow s.v < t.v$ .

$$(\Rightarrow) s \rightarrow t \Rightarrow s.v < t.v$$

Case 1:  $s.p = t.p$  As we can see from the vector clock algorithm, within the same thread, each event will first set its new vector clock to be the max of previous vector clock and vector on object. Thus  $s.v < t.v$ . Case 2:  $s.q = t.q$ .  $s$  immediately precedes  $t$  in the sequence of events on object  $s.q$ . Therefore,  $s.v[s.q] = t.v[s.q] + 1$  and for  $\forall j \neq s.q: s.v[j] = t.v[j]$ . Thus,  $s.v < t.v$ .

$$(\Leftarrow) s \rightarrow t \Rightarrow s.v \not< t.v$$

From Lemma 2, we have  $s \rightarrow t \Rightarrow t.v[s.c] < s.v[s.c]$ . Thus,  $s.v \not< t.v$ . ■

**Theorem 3: (Optimality:)** The *mix-vector-clock* is optimal in size.

**Proof:** The offline algorithm makes use of the components of minimum vertex cover  $v$  of the thread-object bipartite graph as the *mix-vector-clock*, which is also a valid vector clock of the computation. We need to show this  $v$  is minimum in size. Since timestamping all events in a computation requires the vector clock being able to order each event. Suppose there exists a smaller vector clock  $v'$  which timestamps all events. The fact that events in computation corresponds to edges in its bipartite graph indicates that  $v'$  is also a vertex cover of the bipartite graph, thus contradicts  $|v| \leq |v'|$ . So, the *mix-vector-clock* obtained by the offline algorithm is optimal in size. ■

#### IV. MIX-VECTOR-CLOCK FOR ONLINE COMPUTATION

In this section, we consider the case when the computation is not given in advance; instead, each event of the computation is revealed in an online fashion. We assume that only one event is revealed at any time. Thus, in the online setting, we need to maintain a valid dynamic vector clock when events of a computation arrive one at a time. The thread-object bipartite graph for the computation changes when events arrive in the online fashion. When an event  $e = (t, o)$  is revealed, there could be two cases. Either the thread  $t$  has already performed some operation on object  $o$ , i.e., there is already an edge between  $t$  and  $o$  in the current thread-object bipartite graph. In this case, the thread-object bipartite graph does not change. For the other case, thread  $t$  has never performed any operation on object  $o$ , i.e., there is no edge between  $t$  and  $o$  in the current thread-object bipartite graph. In this case, an edge between  $t$  and  $o$  should be added into the thread-object bipartite graph. Note that in the online setting the idea of using minimum vertex cover to be the components of mix-vector-clock cannot be applied, since the minimum vertex cover of the thread-object graph is changing and the existing components in a mix-vector-clock should not be modified as a new event arrives. That is, we can only add new components to the vector clock.

The conventional naive solution is to always choose thread or always choose object as components of the vector clock as a new event occurs. This mechanism would result in a vector clock with size equal to the number of threads or objects for all computations. Another intuitive mechanism is to randomly choose the object or the thread to add into the vector clock with equal probability. Notice that the hardness of timestamping an online computation stems from the unpredictability of future events. We can only estimate the future using information we already have. Therefore, we propose another mechanism which makes use of the partial computation occurred so far to predict the future events. Specifically, when a new event occurs, if the associated object is more "popular" than the associated thread, then we choose the object, otherwise, we choose the thread. We propose the definition of popularity as below.

**Definition 1:** The *popularity* of a vertex  $v$  in a bipartite graph  $G = \{U \cup V, E\}$  is  $pop(v) = \frac{deg_v}{|E|}$ , where  $deg_v$  is the degree of vertex  $v$  and  $|E|$  is the total number of edges in the graph. We say one node is more popular than another node if it has higher *popularity*.

We assume that only one event can occur at a single time. An event comes in with its associated thread and object. We are not supposed to modify components existing in the mix-vector-clock. The three mechanism are formally listed as:

1. *Naive*: Always choose threads or objects.
2. *Random*: Randomly choose the associated object or thread of the new event with equal probability.
3. *Popularity*: Based on popularity of threads and objects. When a new event comes in. If one of the associated thread or object is already in the vector clock, the the vector clock remains same. Otherwise, compute the popularity of the associated thread and object, add the one with higher popularity into the vector clock.

## V. EVALUATION

In this section, to evaluate the performance of our offline algorithm and compare the performance of the three mechanisms we proposed for online setting, we consider the following two Scenarios:

*Uniform*: Evaluating on a uniformly and randomly generated thread-object bipartite graph, i.e, each thread and object have same popularity.

*Nonuniform*: Evaluating on a thread-object bipartite graph in which a small fraction of objects and threads are much more popular than other threads and objects.

The bipartite graph in *Uniform* scenario could be generated by adding a edge between each thread and each object with a same specific probability. For *Nonuniform* scenario, the bipartite graph can be generated by adding an edge between popular threads and objects with a higher probability and non-popular threads and objects with a smaller probability.

In our first evaluation we consider how graph density affects the vector clock size of the three mechanisms. We set the number of threads and objects in the computation to be 50, respectively, i.e, each side of the thread-object bipartite graph has 50 nodes. We compute the final vector clock size

by applying the above three mechanisms to the above two different scenarios, as the density of thread-object bipartite graph increases. The results are shown in Fig. 4. The first important conclusion we get is that when the density of the bipartite graph is small *Random* and *Popularity* method produce significantly smaller vector clock size than the *Naive* method. However, when the density of graph exceeds a certain threshold, their performance becomes worse than *Naive*. In addition, we found that *Random* and *Popularity* mechanism can obtain much better solution in the *Nonuniform* case than the *uniform* case. Thus, we can conjecture that those two methods are better suited in the computation in which some objects or threads are more popular than other objects and threads. Comparing performance of *Random* and *Popularity*, we found *Popularity* is a slightly better than *Random*. This can be explained by the fact that by choosing popular nodes as vector clock component, we can cover more edges. Thus, the vector clock size would be smaller.

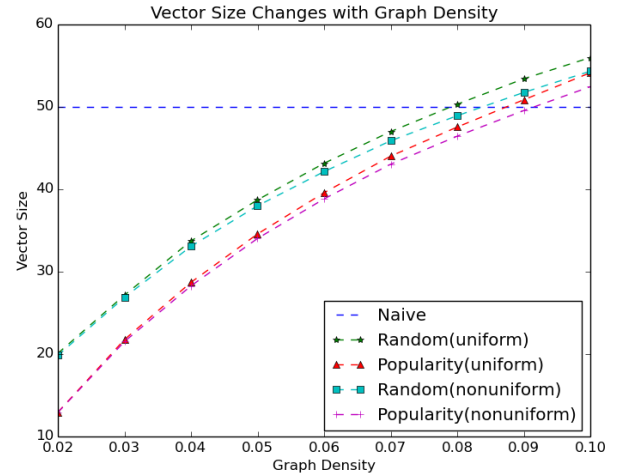


Fig. 4. Vector Size Varies as Graph Density Increases.

In our second evaluation, we fix the density of graph to be 0.05 and evaluate the performance of the three mechanisms as we increase the number of nodes in the thread-object bipartite graph. From Fig. 5, we can notice that as the number of nodes in the graph increases, i.e, the number of threads and objects increases in a computation, the vector size increases. When the number of nodes is below a certain threshold, 70 here, *Random* and *Popularity* generates smaller vector clock size than *Naive*. Once the number of nodes exceeds that threshold, *Naive* is better, which means by simply choosing either all threads or objects as vector clock components gives a smaller vector clock. Therefore, we can get the conclusion that these two techniques are more effective in simple computations, i.e, in computations which involves relatively small number of threads and objects.

In our third evaluation, we want to know how far the online case drifts from the static case. We choose the *Popularity* mechanism for the online case. We also consider the *Naive*

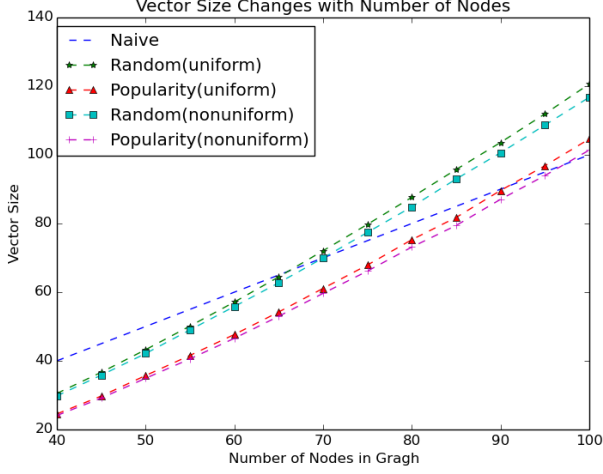


Fig. 5. Vector Size Varies as Number of Nodes Increases

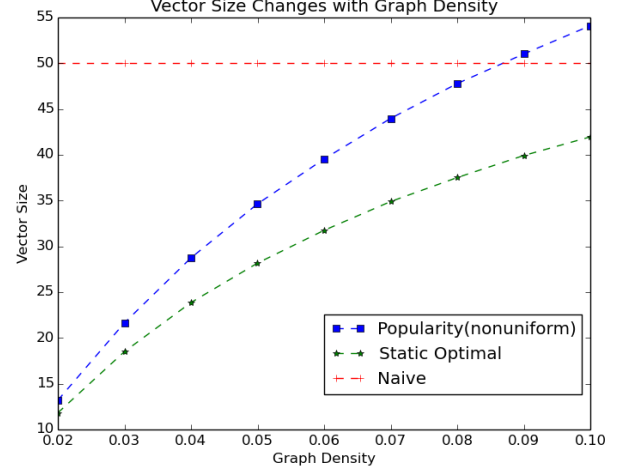


Fig. 6. Vector Size Varies as Graph Density Increases

mechanism, which can both be applied to online case and static case and generates the same vector clock. For the static case, we use our offline algorithm proposed in Section III. For the experiment, we first apply the *Popularity* mechanism as we reveal the edge of the graph one by one. Then, after we have the whole graph, we apply the offline algorithm. Fig. 6 shows the results we get when we set the number of nodes to be 50 and increase graph density. Fig. 7 shows the results we get when we fix the density to be 0.05 and increase the number of nodes in the graph. First, we can notice that our offline algorithm generates a vector clock with significantly smaller size than the *Naive* solution. For example, *Naive* has a vector clock of size 50 and the offline algorithm reduces that to be around 35, when there are 50 threads and the graph density is 0.05. Besides, in online setting, although the *Popularity* mechanism cannot completely achieve as small vector size as the optimal solution does, the gap is within a reasonable number. For example, *Popularity* generates a vector of size round 56 while the optimal is round 48, when there are 70 threads and the graph density is 0.05. Also, as graph density or number of nodes in graph increases, the gap is increasing which again indicates the *Popularity* mechanism is not suitable for relatively dense graph.

Combining the results of the above evaluations, for the online problem, a practical mechanism would be to set thresholds for both graph density and number of nodes in graph. At the beginning, we adopt the *Popularity* mechanism and as more events come in we adopt the *Naive* approach if the graph parameters exceeds the thresholds.

## VI. RELATED WORK

Several techniques have been proposed to reduce the overhead imposed by Fidge/Mattern's vector clocks [4]–[6], [11]. Singhal and Kshemkalyani [12] present a technique to reduce the amount of data piggybacked on each message. The main idea is to only send those entries of the vector along with

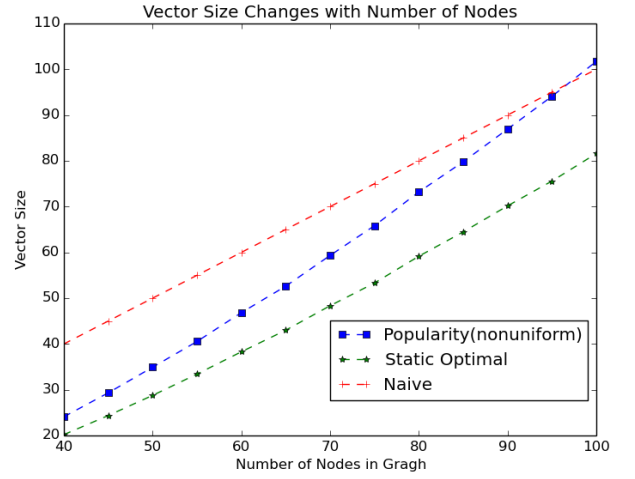


Fig. 7. Vector Size Varies as Number of Nodes Increases

a message that have changed since a message was last sent to that process. H  lary *et al* [8] further improve upon Singhal and Kshemkalyani technique and describe a suite of algorithms that provide different trade offs between space overhead and communication overhead. The ideas described in the two papers are actually orthogonal to the ideas presented in this paper and, therefore, can also benefit our timestamping algorithm by reducing its overhead.

Agarwal and Garg [2] have proposed a class of logical clock algorithms, called chain clock, for tracking dependencies between *relevant* events based on generalizing a process to any chain in the computation poset. Their algorithm reduces the number of components required in the vector when the set of relevant events is a small fraction of the total events. Our work is not closely related to this work. They provide two different algorithms: the first algorithm adds any newly arrived event to a chain with the guarantee that no more that

$|P|$  chains are necessary. The second algorithm uses online chain decomposition of a poset to guarantee that no more than  $(w+1)w/2$  chains are necessary where  $w$  is the width of the poset. In this paper, our algorithm uses components that are either for the process or for the object and guarantees that the number of components is never more than  $\min(|P|, |Q|)$ .

Garg, Skawratananond, and Mittal [7] have proposed an algorithm to timestamp messages in a distributed system. They assume that all messages are synchronous and show that such systems can have timestamps of vector clocks with dimension less than  $N$ . They define the notion of an undirected communication graph with the set of vertices as processes and the edges denoting which processes can communicate. They show that the number of components required is equal to the number of stars and triangles the communication graph can be decomposed into. Our technique of using vertex cover is inspired from that work even though their work is strictly for distributed systems and they do not consider mixed-clocks. We have two types of entities in our system — threads and objects and the dimension of the vector clock reduces to a vertex cover of the bipartite graph that represents the interaction between these entities.

## VII. CONCLUSIONS

This paper proposes an offline algorithm to compute a mixed vector clock composed of a mix of threads and objects, which is shown to be a correct vector clock and have optimal size, to timestamp events in a computation. Thread-object bipartite graph is constructed based on the given computation and then the minimum vertex cover of this bipartite graph is computed and the threads and objects in this vertex cover are adopted as the components of the mix vector clock. In an online computation in which events are coming one by one, two mechanisms are proposed, the *Random* mechanism which randomly choose the associated thread or object as vector clock component and the *Popularity* mechanism which choose the thread or object based on their popularity. By evaluating on thread-object bipartite graphs with different characteristics, we get the conclusion that the *Popularity* mechanism shows best performance on *non-uniform* graphs.

## REFERENCES

- [1] Bondy J. A. and Murty U. S. R. *Graph Theory with Applications*. North Holland, 1976.
- [2] A. Agarwal and V. K. Garg. Efficient Dependency Tracking for Relevant Events in Shared-Memory Systems. In *Proceedings of the, ACM Symposium on Principles of Distributed Computing (PODC)*, pages 19–28, 2005.
- [3] Hopcroft John E. and Richard M. Karp. An  $n^{5/2}$  algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–31, Dec 1973.
- [4] C. J. Fidge. Timestamps in Message-Passing Systems that Preserve the Partial-Ordering. In K. Raymond, editor, *Proceedings of the 11th Australian Computer Science Conference (ACSC)*, pages 56–66, February 1988.
- [5] C. J. Fidge. Partial Orders for Parallel Debugging. In *Proceedings of the ACM/ONR Workshop on Parallel and Distributed Debugging*, pages 183–194, January 1989.
- [6] C. J. Fidge. Logical Time in Distributed Computing Systems. *IEEE Computer*, 24(8):28–33, August 1991.

- [7] Vijay K. Garg, Chakarat Skawratananond, and Neeraj Mittal. Timestamping messages and events in a distributed system using synchronous communication. *Distributed Computing*, 19(5-6):387–402, 2007.
- [8] J.-M. Hélary, M. Raynal, G. Melideo, and R. Baldoni. Efficient Causality-Tracking Timestamping. *IEEE Transactions on Knowledge and Data Engineering*, 15(5):1239–1250, 2003.
- [9] L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM (CACM)*, 21(7):558–565, July 1978.
- [10] K. Marzullo and L. Sabel. Efficient Detection of a Class of Stable Properties. *Distributed Computing (DC)*, 8(2):81–91, 1994.
- [11] F. Mattern. Virtual Time and Global States of Distributed Systems. In *Parallel and Distributed Algorithms: Proceedings of the Workshop on Distributed Algorithms (WDAG)*, pages 215–226. Elsevier Science Publishers B. V. (North-Holland), 1989.
- [12] M. Singhal and A. Kshemkalyani. An Efficient Implementation of Vector Clocks. *Information Processing Letters (IPL)*, 43:47–52, August 1992.